CAREERAI — AN AI-POWERED ONE-STOP PERSONALIZED CAREER & EDUCATION ADVISOR

A MINI PROJECT REPORT

Submitted by

Sri Sowbarniga K
REG.NO:9222524106207

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMUNICATION ENGINEERING

V.S.B.ENGINEERING COLLEGE (AUTONOMOUS)

KARUR-639111.

JULY 2026

V.S.B.ENGINEERING COLLEGE (AUTONOMOUS)

KARUR-639111.

BONAFIDE CERTIFICATE

Certified that this project report **"CareerAI — An AI-Powered One-Stop Personalized Career & Education Advisor"** is the bonafide work of

Sri Sowbarniga K, Reg. No. **[922524106207]** who done this project.

.

SIGNATURE OF THE FACULTY **SIGNATURE OF THE HOD**

CAREERAI — AN AI-POWERED ONE-STOP PERSONALIZED CAREER & EDUCATION ADVISOR

Abstract

Students today face a fragmented career-planning experience — career guidance, resume building, skill assessment, college search, and exam tracking are all scattered across different tools and platforms, with nothing tying them together around a student's actual academic profile. This project presents CareerAI, a full-stack web application built to act as a personalized guide for students by unifying seven career-planning functions under one authenticated platform: AI-driven career recommendation, career roadmap generation, an AI chatbot counsellor, a multi-template AI resume generator, a college finder, skill gap analysis, and an exam tracker.

The system is built using React.js for the frontend, Node.js and Express.js for the backend, MongoDB for persistent user data, and Groq's LLaMA 3.3-70B model for AI-generated content. Users register and build a profile — grade, marks, subjects, interests, and skills — and the system uses this profile to generate personalized career matches with percentage fit and reasoning, a step-by-step roadmap toward a chosen career, a professional resume in a selectable template, a ranked list of Indian colleges, and a skill-gap checklist comparing the student's current skills against what a target career actually requires.

The application is deployed on Vercel with MongoDB Atlas as the cloud database, making it publicly accessible at near-zero operating cost. Testing across all seven modules confirmed reliable AI response parsing, stable authentication, and consistent recommendation quality, with observed cold-start latency on the free-tier backend as the main limitation. By consolidating previously scattered career-guidance touchpoints into one AI-personalized dashboard, CareerAI demonstrates a practical, extensible response to the need for student-specific, data-driven career guidance, with clear scope for further enhancement such as persistent exam-tracking and real-time job market integration.

INTRODUCTION

Career decision-making is one of the most consequential choices a student makes, yet the tools available to support that decision are scattered across many disconnected sources. A student looking for career direction typically has to visit one website for a generic career-interest quiz, another for resume-writing tips, manually research colleges through word-of-mouth or scattered rankings, and separately track entrance exam dates through personal notes or spreadsheets. None of these tools adapt to the student's actual academic record, current skill level, or personal interests — they offer the same generic advice to every user regardless of individual circumstances.

This lack of personalization becomes especially problematic in fast-evolving, skill-driven fields such as technology, where the specific competencies required for a role change frequently and a student may not even know which skills they are missing until they attempt to apply for that role. Traditional academic career counselling, where available, relies on human counsellors who cannot scale to give every student individualized, continuously updated attention.

At the same time, recent advances in large language models (LLMs) have made it possible to generate natural-language, context-aware recommendations cheaply and at scale. An LLM can be given a student's specific profile — their marks, subjects, interests, and current skills — and asked to generate personalized career suggestions, a structured learning roadmap, or a skill-gap analysis, in a way that static, rule-based systems cannot.

This project addresses the fragmentation problem by designing and building CareerAI, a single web platform that acts as a personalized guide for students. Instead of visiting multiple disconnected tools, a student creates one profile and gains access to seven integrated functions — career recommendation, career roadmap generation, an AI chatbot, a resume generator, a college finder, skill gap analysis, and an exam tracker — all powered by a single large language model backend and tied to their authenticated account.

1.1 Problem Identified

- No single platform ties career guidance, resume building, college search, skill assessment, and exam tracking together
- Career advice available online is generic and does not consider a student's actual marks, subjects, interests, or skills
- Students have no easy way to identify exactly which skills they are missing for a specific target career
- No conversational, on-demand guidance channel exists for students to ask quick career-related questions

1.2 Objectives of the Project

- To build a single, authenticated web platform that acts as a personalized guide, combining career guidance, skill assessment, and educational planning
- To use a large language model (LLaMA 3.3-70B, accessed via the Groq API) to generate personalized career recommendations, roadmaps, resumes, college lists, chatbot replies, and skill-gap reports based on each student's own profile
- To persist student profile and resume data securely in MongoDB, tied to a JWT-authenticated account
- To deploy the platform publicly on Vercel, so it is accessible to any student at near-zero cost

1.3 Scope of the Project

The project covers seven integrated modules — Career Recommendation, Career Roadmap, AI Chatbot, Resume Generator (with three switchable templates), College Finder, Skill Gap Analysis, and Exam Tracker — built around a single MongoDB-backed user profile and secured with token-based authentication.

1.4 Motivation

The motivation for this project comes from a simple, common observation: every student eventually needs to answer the same set of questions — which career suits me, what am I missing to get there, how do I present myself on paper, where should I study, and when are my exams — yet no single, accessible tool answers all of them together using the student's own data. With large language models now capable of generating coherent, context-aware, natural-language guidance cheaply and quickly, it became practical to build a system that treats these five questions as one connected problem rather than five separate tools.

1.5 Organization of the Report

This report is organized as follows. The Introduction outlines the problem, objectives, and scope. Contribution of the Work summarizes the individual effort behind the project. The Literature Survey reviews related work in AI-based career guidance and resume analysis. System Analysis compares the existing and proposed systems and studies feasibility. Proposed Work and the CareerAI Workflow describe how the system operates end to end. System Design and System Implementation cover the architecture, database, API design, and module-by-module implementation details. Testing documents the verification approach and sample test cases. Results and Discussion presents the observed behaviour of the completed system. Finally, the Conclusion summarizes the outcomes and lists Future Enhancements, followed by References and an Appendix containing sample code.

2. CONTRIBUTION OF THE WORK

This project was carried out as an individual effort, covering the complete design, development, and deployment of the CareerAI platform.

- Identified the problem of scattered, disconnected career-guidance resources and designed a single platform that unifies career recommendation, skill gap analysis, resume building, college search, an AI chatbot, and exam tracking around one student profile.
- Designed the database structure — a single User collection with embedded profile and resume sub-documents — and planned the complete set of REST API endpoints connecting the frontend and backend.
- Built the entire Node.js/Express.js backend, including user registration and login with bcrypt password hashing and JWT-based authentication, and every API route needed for the seven modules.
- Integrated the Groq API with the LLaMA 3.3-70B model, designed the prompts used for each AI feature, and implemented logic to reliably extract structured JSON from the AI's raw text response, with fallback handling when a response doesn't parse cleanly.
- Built the entire React frontend as a single-page application — login/register, profile form, career results, roadmap view, chatbot, resume generator with three templates and inline editing, college finder, dashboard, skill gap checklist, and exam tracker — and connected every screen to the backend via axios, including retry logic to gracefully handle backend cold-start delays.
- Tested each module individually and as a complete end-to-end flow, then deployed the frontend and backend separately on Vercel, with MongoDB Atlas as the cloud database.
- Prepared the complete project documentation, including system design, module description, database structure, and testing results.

3. LITERATURE SURVEY

Ref	Author (Year)	Methodology	Results Achieved	Limitations
[1]	Bahalkar, Peddi & Jain (2024)	AI-driven career guidance predicting career paths using academic performance, skills, and interests via a Random Forest classifier	Achieved 88% accuracy with ranked recommendations and required-skill listings	Limited to classical ML models; needs NLP and live job-market analytics
[2]	IEEE Xplore Study (2025)	Combined the RIASEC interest model with Decision Trees, XGBoost, and SVM for student career pathway prediction	Improved personalization and scalability of career recommendations	Static interest-category matching, with no real-time adaptation
[3]	Mirajkar, Shinde, Shelke, Yadav & Shinde (2024)	Combined skill recommendation and resume analysis system using AI techniques	Enabled automated skill recommendation alongside resume evaluation	Limited detail on scalability across diverse job roles
[4]	Research Digest, Engineering Management (2026)	Dual-mode LLM-based resume intelligence system for recruiters and job seekers, using context-aware parsing and vector search	Surfaces resume insights, flags skill gaps, and generates improvement guidance	Traditional keyword approaches it compares against give little candidate feedback
[5]	Sharma, A. (2024)	Transformer-based contextual embeddings comparing job-description requirements against resume content	Correctly matches semantically related skills expressed with different wording	Not peer-reviewed; lacks large-scale quantitative evaluation
[6]	IEEE Xplore (2024)	LLM-based resume building application with Generation, Assessment, and User I/O modules via prompt engineering	Effective, easy-to-use resume drafting aimed at students	Focused only on resume drafting, not broader career guidance
[7]	Tavakoli, Faraji, Vrolijk, Molavi, Mol & Kismihók (2022)	Built an open recommender system linking labor-market data with personalized education recommendations	Delivered labor-market-driven course and skill recommendations at scale	Depends heavily on the quality and freshness of external labor-market datasets
[8]	Gadegaonkar, Lakhwani, Marwaha & Salunke (2023)	Applied machine learning techniques to match candidate profiles with suitable job openings	Improved matching accuracy between candidate skills and job requirements	Limited to job matching; does not cover resume building or skill-gap identification

3.1 Summary of Literature Survey

- Existing work splits into two tracks: classical ML-based career predictors, which are accurate but static and don't adapt to a student's changing skill level, and newer LLM-based resume and skill-gap tools, which understand context well but are largely recruiter-facing or limited to resume drafting alone. None of the reviewed systems combine career recommendation, roadmap planning, resume generation, college search, skill gap analysis, and exam tracking into one personalized, authenticated platform — the exact gap CareerAI is built to close.

4. SYSTEM ANALYSIS

4.1 Existing System

Career guidance today is typically delivered through one of a few disconnected channels: human counsellors, who cannot scale to give every student individualized and continuously updated attention; static online quizzes and checklists that offer the same generic advice regardless of a student's actual profile; standalone resume-writing guides with no connection to career planning; and manual, self-maintained tracking of entrance exam dates and college research.

4.2 Limitations of the Existing System

- No personalization based on the student's real academic marks, subjects, interests, or skills
- No single system connects career guidance, resume building, college search, and exam tracking
- No continuous, on-demand feedback loop for quick career questions
- No modern AI/LLM capability used to generate adaptive, natural-language guidance

4.3 Proposed System

CareerAI addresses these limitations by combining a conversational AI layer (LLaMA 3.3-70B, accessed via the Groq API) with structured recommendation logic across seven modules, all built around a single authenticated student profile stored in MongoDB. Table 4.1 compares the existing and proposed systems across key aspects.

Aspect	Existing System	Proposed System (CareerAI)
Career guidance	Generic, one-size-fits-all quizzes and articles	AI-generated recommendations personalized to the student's own profile
Skill assessment	Manual self-assessment or none at all	AI-driven skill gap checklist against a specific target career
Resume building	Separate resume-writing tools or manual drafting	AI-generated resume with 3 templates and inline editing
College search	Word-of-mouth or scattered directory websites	AI-ranked college list based on grade, marks, interests, and state

Exam tracking	Manual notes or spreadsheets	Centralized exam list with category filtering
Data continuity	No shared profile across tools	One authenticated profile feeds every module

4.4 Feasibility Study

Technical feasibility: the platform is built entirely on well-documented, free or low-cost tools — React, Node.js, MongoDB Atlas's free tier, Groq's free API tier, and Vercel's free hosting tier — all of which are mature and widely supported.

Economic feasibility: at student-project scale, the platform runs at near-zero operating cost, since every service used offers a sufficient free tier for development and demonstration purposes.

Operational feasibility: the platform requires no technical training to use — a single login gives access to all seven modules through a simple, consistent navigation bar.

5. PROPOSED WORK

CareerAI works as a connected sequence of AI-assisted steps built around one authenticated student profile. To begin with, a student registers and logs in, after which their password is hashed and a JWT session token is issued. The student then fills in a profile form — grade, marks, strong subjects, interests, and current skills — which becomes the shared input for every downstream module.

Once the profile is available, the student can request career recommendations: the backend sends the profile to the Groq API running the LLaMA 3.3-70B model, which returns the top three matching careers, each with a percentage match score and a reason. Selecting any one of these careers triggers a roadmap request, where the same LLM generates a five-step, time-boxed learning path toward that career.

In parallel, the student can use the AI chatbot for free-text career questions, generate an AI-written resume across three visual templates (which can be edited inline and saved back to the database), search for a ranked list of colleges based on their grade, marks, interests, and state, and run a skill gap analysis for any target career — in which the LLM lists eight essential skills for that career and marks which ones the student already has, based on their stated skill set. An exam tracker module rounds out the platform, listing major Indian entrance exams with category-based filtering.

Every AI-generated response is returned by the LLM as free text but instructed, through prompt design, to follow a specific JSON structure; the backend extracts this JSON using pattern matching before sending it to the frontend, with a fallback message shown if the response cannot be parsed.

5.1 Advantages of the Proposed System

- Personalized guidance generated from the student's own marks, subjects, interests, and skills, instead of generic, one-size-fits-all advice
- Seven previously disconnected career-planning functions unified under a single authenticated profile
- Fast, natural-language recommendations powered by a large language model rather than static rule tables
- Editable, savable AI-generated resumes across multiple visual templates
- Runs entirely on free-tier cloud services, keeping operating cost near zero at student-project scale

6. CAREERAI WORKFLOW

Student Registration & Login

↓

Profile Creation (grade, marks, subjects, interests, skills)

↓

AI Career Recommendation (Groq — LLaMA 3.3-70B)

↓

Career Roadmap Generation

↓

Parallel Modules — AI Chatbot | Resume Generator | College Finder | Skill Gap Analysis | Exam Tracker

↓

MongoDB Persistence (profile & resume)

↓

Dashboard (consolidated view of all data)

The workflow is designed so that the student's profile, captured once at the start, feeds every AI-driven module downstream, avoiding repeated data entry and keeping every recommendation grounded in the same, real student data.

7. SYSTEM DESIGN

7.1 System Architecture

The platform follows a three-tier architecture. The presentation layer is a React single-page application that switches between views (Login, Home, Profile, Result, Roadmap, Chatbot, Resume, Colleges, Dashboard, Skill Gap, Exams) using component state rather than a router library. The application layer is a Node.js/Express.js REST API, deployed on Vercel, that handles authentication, profile and resume persistence, and orchestrates all calls to the Groq LLM API. The data and AI layer consists of MongoDB Atlas for persistent user data and the Groq API running the LLaMA 3.3-70B model for all AI-generated content.

7.2 Database Design

The application uses a single MongoDB collection, User, with embedded sub-documents for profile and resume data, as shown below:

Field	Type	Notes
name, email, password	String	password bcrypt-hashed
profile.grade / marks / subjects / interests / skills	String	student profile inputs
resume.objective / education / template	String	AI-generated resume fields
resume.skills / interests / projects	[String]	array fields in generated resume
createdAt	Date	auto timestamp

Known limitation: *no savedExams field currently exists — the exam "save" action is frontend-only state, not persisted to the database (see Section 11.2, Future Enhancement 1).*

6.3 API Endpoint Design

Endpoint	Method	Purpose
/register, /login	POST	JWT-based authentication, bcrypt password hashing
/save-profile	POST	Persist student profile to MongoDB
/save-resume	POST	Persist AI-generated (and user-edited) resume
/career-recommend	POST	Groq LLM call, top 3 career matches with % and reason

/career-roadmap	POST	Groq LLM call, 5-step roadmap with durations
/chatbot	POST	Groq LLM call, conversational career-guidance reply
/generate-resume	POST	Groq LLM call, structured JSON resume

8. SYSTEM IMPLEMENTATION

8.0 Software and Hardware Requirements

Software requirements: Node.js runtime and npm for the backend; a modern web browser (Chrome/Edge/Firefox) for the frontend; MongoDB Atlas as the cloud database; a Groq API key for LLM access; and a Vercel account for deployment of both frontend and backend.

Hardware requirements: any development machine capable of running Node.js and a browser (a standard laptop/desktop with at least 4 GB RAM is sufficient for development); no special hardware is required for end users, since the application runs entirely as a web app in the browser and all AI processing happens on Groq's servers rather than the client device.

8.1 Technology Stack

The frontend is built with React.js. The backend uses Node.js with Express.js. MongoDB (via Mongoose) provides persistent storage. Authentication uses bcryptjs for password hashing and jsonwebtoken for session tokens. AI features use the Groq SDK with the llama-3.3-70b-versatile model. HTTP communication between frontend and backend uses axios. Both frontend and backend are deployed on Vercel.

8.2 Authentication Module

Registration checks for an existing email, hashes the password with bcryptjs (10 salt rounds), and issues a 7-day JWT signed with a secret key. Login verifies the password using bcrypt.compare. The frontend implements a retry loop of up to five attempts, three seconds apart, to gracefully handle cold-start delays on the free-tier backend, informing the user that the server is waking up.

8.3 Career Recommendation and Roadmap Module

The career-recommend endpoint sends the student's profile to the Groq LLM and returns three career suggestions with percentage match and reasoning. Selecting a career calls the career-roadmap endpoint, which generates a five-step, time-boxed roadmap toward that career.

8.4 AI Chatbot Module

The chatbot endpoint accepts a free-text question along with the student's name and grade, and returns a concise two-to-three sentence counselling reply, rendered as a chat conversation thread in the frontend.

8.5 Resume Generator Module

The generate-resume endpoint returns structured JSON containing an objective, education summary, skills, interests, and projects. This data is rendered using three selectable visual templates — Modern, Classic, and Minimal — all built from the same underlying data. An inline edit mode lets students revise the AI-generated text directly before saving it back to MongoDB via the save-resume endpoint, and a print-to-PDF option is provided using the browser's native print function.

8.6 College Finder Module

The college-recommend endpoint accepts the student's grade, marks, interests, and state, and returns five ranked Indian colleges with type, course, and cutoff information.

8.7 Skill Gap Analysis Module

The skill-gap endpoint accepts a target career and the student's current skills, and returns eight essential skills for that career, each flagged as already possessed or missing. The frontend renders this as a checklist with a summary count of skills already held.

8.8 Exam Tracker Module

The exams endpoint returns a static list of ten major Indian entrance exams — JEE Main, JEE Advanced, NEET, CUET, CLAT, CAT, GATE, NDA, BITSAT, and VITEEE — with category-based filtering. A "Save Exam" toggle lets students

9. TESTING

9.1 Types of Testing Performed

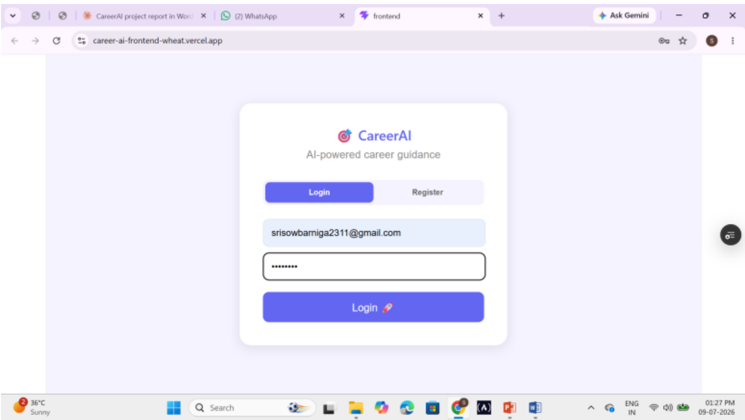
Unit testing was carried out on each API route in isolation. Integration testing verified the complete round trip from frontend, through the backend API, to the Groq LLM and MongoDB, and back. User acceptance testing covered the full profile-to-recommendation flow as a real student would use it.

9.2 Sample Test Cases

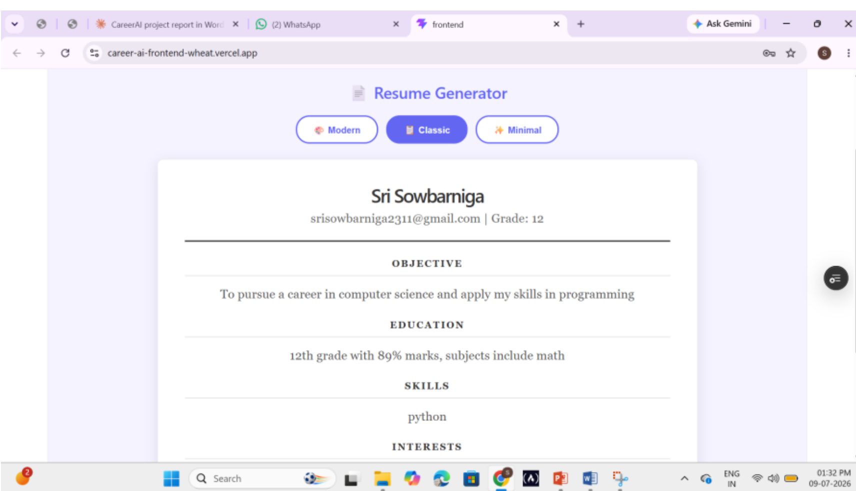
ID	Description	Input	Expected	Status
TC01	Register new user	Valid name/email/password	JWT token + success	Pass
TC02	Duplicate email register	Existing email	"Email already exists!"	Pass
TC03	Wrong password login	Valid email, wrong password	"Wrong password!"	Pass
TC04	Career recommend	Filled profile form	3 careers with % match	Pass
TC05	Skill gap analysis	Target career + skills	8-skill checklist	Pass
TC06	Save exam, refresh page	Click Save Exam then reload	Exam remains saved	Fail (not persisted)

10. RESULTS AND DISCUSSION

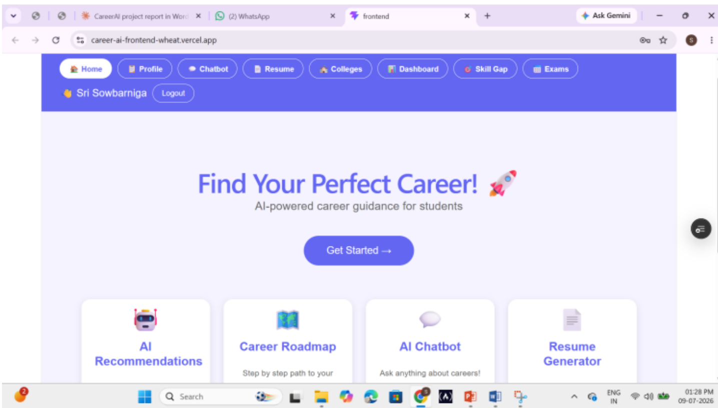
10.1 Output Screens



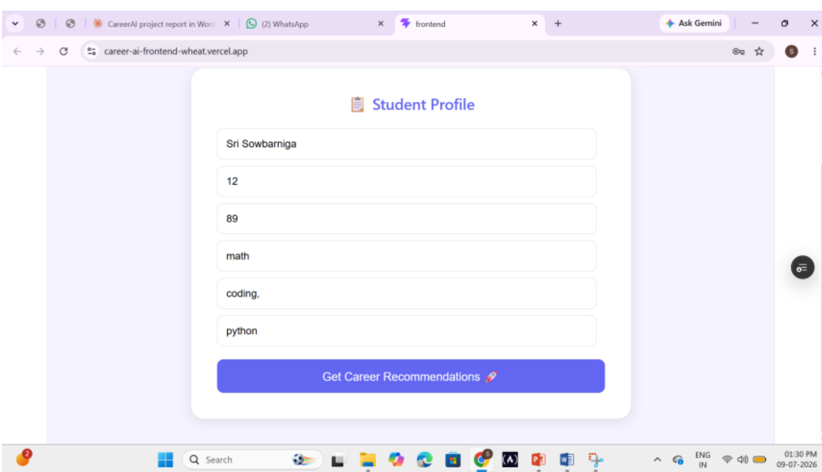
Login/Register screen: presents a toggle between Login and Register modes, with name, email, and password fields, and a loading state that informs the user if the free-tier backend server is starting up.



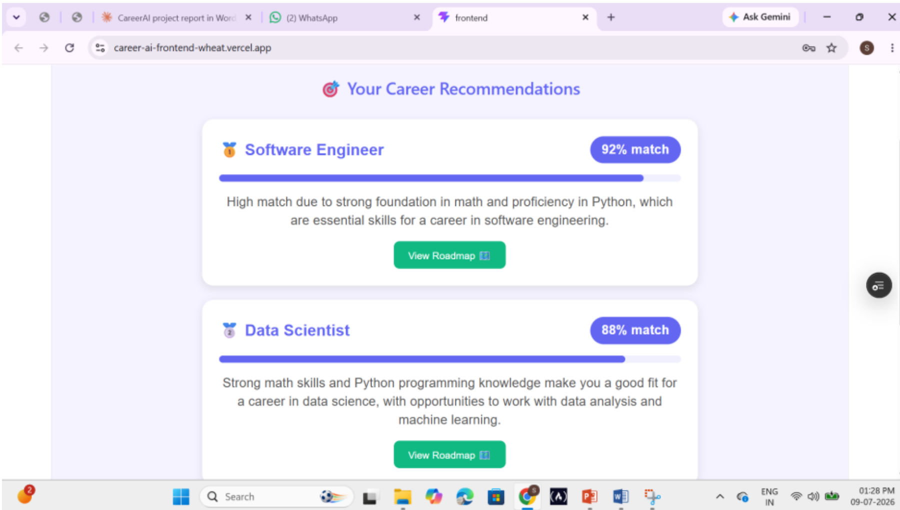
Resume screen: lets the student pick from three templates (Modern, Classic, Minimal), view the AI-generated resume, switch to an inline edit mode, save the edited resume, or print/export it as a PDF.



Home page: displays a welcome banner and a grid of seven feature cards — one for each module — giving the student a single entry point into the platform.

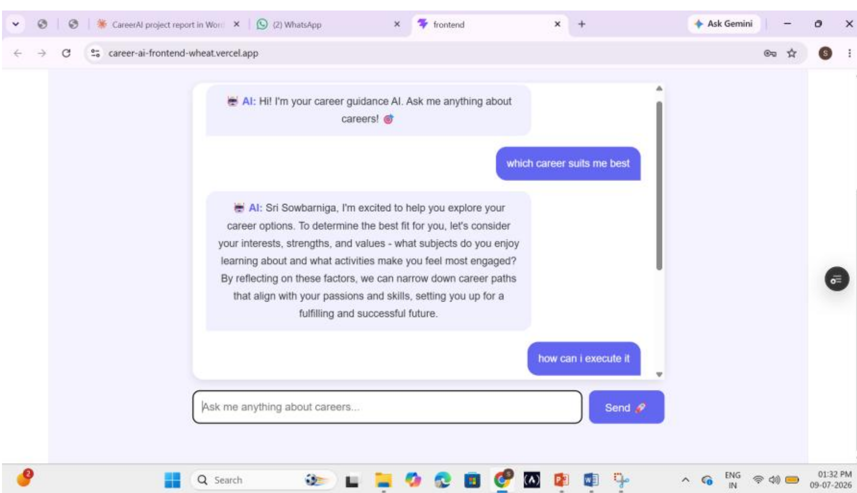


Profile form: collects the student's name, grade, marks, strong subjects, interests, and current skills, which are then used as the shared input for every AI-driven module.



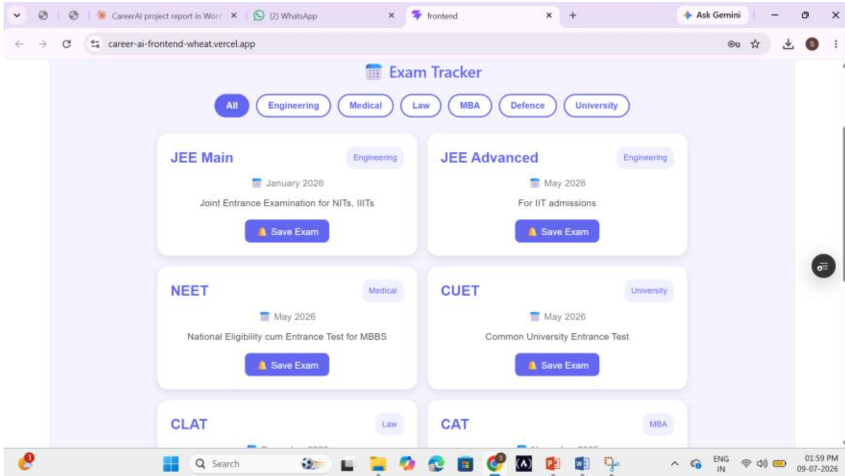
Career Result cards: display the top three AI-recommended careers, each with a percentage match shown as a progress bar and a short reason, along with a button to view the roadmap for that career.

Roadmap timeline: shows a five-step, numbered roadmap toward the selected career, each step paired with an estimated time duration

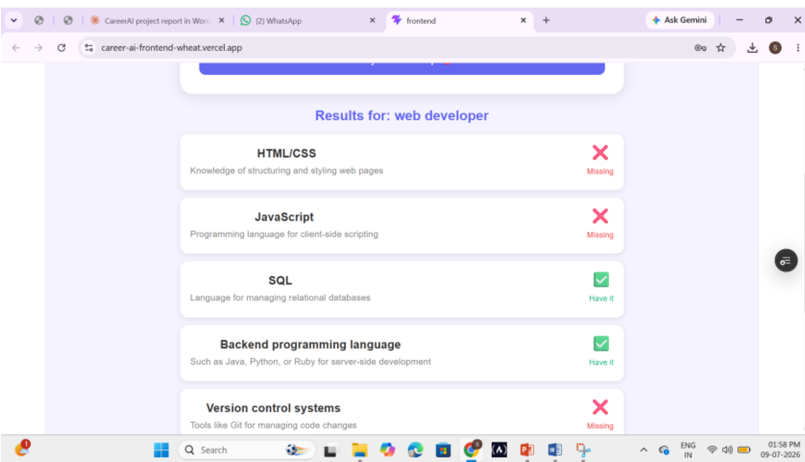


.

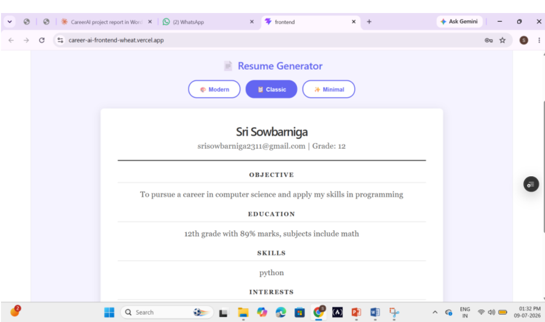
Chatbot conversation: a chat-style interface where the student's questions and the AI's replies are shown as alternating message bubbles



Exam Tracker grid: displays ten major entrance exams with category filter buttons and a "Save Exam" toggle for building a personal shortlist



Skill Gap checklist: lists eight skills required for the target career, each marked with a ✅or ❌icon, along with a summary count of skills already held.



Dashboard: consolidates the student's name, email, grade, and marks alongside quick-access cards for all seven modules.

10.2 Discussion

The completed CareerAI platform was tested by using it as an actual student would: registering an account, filling in a sample profile, and running each of the seven modules in sequence.

Authentication worked reliably, with bcrypt-hashed passwords and JWT tokens issued correctly. The frontend's retry logic successfully handled backend cold-start delays on the free-tier Vercel deployment, showing the user a "server waking up" message instead of failing outright.

Given a sample profile, the career recommendation module consistently returned three distinct career suggestions with plausible percentage-match scores and relevant reasoning, and generated a coherent five-step roadmap for the selected career. The chatbot returned concise, relevant responses in most test cases. All three resume templates rendered correctly, and inline editing followed by save-to-database worked as expected. The college finder and skill gap analysis modules both returned well-structured, relevant results across a range of test career inputs.

The exam tracker's list and category filter worked correctly; however, testing confirmed that the "Save Exam" action only updates the frontend's local state and does not persist to the database, meaning saved exams are lost on page refresh. This has been identified as the platform's main current limitation.

Overall, the Groq LLaMA 3.3-70B integration returned structured JSON reliably across repeated test runs, with the regex-based extraction and fallback logic successfully catching the rare malformed response. The main observed friction point across all modules was backend cold-start latency, which was mitigated — but not eliminated — by the retry-with-message approach on the frontend.

11. CONCLUSION AND FUTURE ENHANCEMENTS

11.1 Conclusion

This project set out to solve the problem of fragmented, generic career guidance by building CareerAI, a single platform that acts as a personalized guide for students. By combining career recommendation, roadmap generation, an AI chatbot, a resume generator, a college finder, skill gap analysis, and an exam tracker under one authenticated profile, the system replaces scattered, one-size-fits-all advice with recommendations generated specifically from each student's own marks, subjects, interests, and skills, using the LLaMA 3.3-70B model via the Groq API. Testing across all seven modules confirmed that the platform performs reliably, with the one identified limitation being that saved exam selections are not yet persisted to the database — a straightforward fix that has been documented as the immediate next step.

11.2 Future Enhancements

- Persist Exam Tracker saves — add a savedExams field to the User schema, a save-exam endpoint, and load saved exams into state on login
- Add email/push reminders for saved exam dates
- Implement proper PDF export using a dedicated library instead of the browser's print function
- Add response caching to reduce repeated Groq API latency on cold starts
- Move to refresh-token-based authentication instead of a single long-lived 7-day JWT

12. REFERENCES

[1] P. Bahalkar, P. Peddi, and S. Jain, "AI-Driven Career Guidance System: A Predictive Model for Student Subject Recommendations Based on Academic Performance and Aspirations," Frontiers in Health Informatics, vol. 13, no. 3, pp. 8216–8230, 2024.

[2] "AI-Powered Career Guidance: A Scalable Model for Personalized Recommendations," IEEE Xplore Conference Publication, 2025.

[3] R. Mirajkar, G. R. Shinde, P. M. Shelke, S. Yadav, and B. Shinde, "Skill Recommendation System and Resume Analysis using AI," 2024 IEEE Pune Section International Conference (PuneCon), Pune, India, 2024, pp. 1–3, doi: 10.1109/PuneCon63413.2024.10895306.

[4] "An LLM-Based Resume Intelligence System for Recruiters and Job Seekers," Research Digest on Engineering Management and Social Innovations, 2026.

[5] A. Sharma, "How Large Language Models (LLMs) Can Help Identify Missing Skills in Your Resume," Medium, 2024.

[6] "Resume Building Application based on LLM (Large Language Model)," IEEE Conference Publication, 2024, doi: 10.1109/document/10425602.

[7] M. Tavakoli, A. Faraji, J. Vrolijk, M. Molavi, S. T. Mol, and G. Kismihók, "An AI-based open recommender system for personalized labor market driven education," Advanced Engineering Informatics, vol. 52, 101508, 2022.

[8] "AI Powered Job Recommendation System," IEEE Conference Publication, 2025.

[9] "An Intelligent Career Guidance System using Machine Learning," IEEE Conference Publication, 2021

[10] S. Gadegaonkar, D. Lakhwani, S. Marwaha, and A. Salunke, "Job Recommendation System using Machine Learning," 2023 Third International Conference on Artificial Intelligence and Smart Energy (ICAIS), 2023, doi: 10.1109/ICAIS56108.2023.10073757.

13. APPENDIX – SAMPLE CODE

Skill Gap Analysis endpoint (backend):

```
app.post("/skill-gap", async (req, res) => {
  const { career, currentSkills, grade } = req.body;
  const prompt = "List 8 essential skills required to become a " + career +
    ". For each skill, check if the student already has it based on their
current skills: " +
    currentSkills + ". Return ONLY a JSON array with fields: skill, description,
hasSkill (true/false)";
  const completion = await groq.chat.completions.create({
    messages: [{ role: "user", content: prompt }],
    model: "llama-3.3-70b-versatile",
  });
  const jsonMatch = completion.choices[0].message.content.match(/\[[\s\S]*\]/);
  res.json({ success: true, data: JSON.parse(jsonMatch[0]) });
});
```

User authentication endpoint (backend):

```
app.post("/login", async (req, res) => {
  await connectDB();
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user) return res.json({ success: false, error: "User not found!" });
  const match = await bcrypt.compare(password, user.password);
  if (!match) return res.json({ success: false, error: "Wrong password!" });
  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn:
"7d" });
  res.json({ success: true, token, user: { name: user.name, email: user.email }
});
});
```

User schema definition (MongoDB / Mongoose):

```
const userSchema = new mongoose.Schema({
  name: String,
  email: { type: String, unique: true },
  password: String,
  profile: { grade: String, marks: String, subjects: String, interests: String,
skills: String },
  resume: { objective: String, education: String, skills: [String],
            interests: [String], projects: [String], template: String },
  createdAt: { type: Date, default: Date.now }
});
```

React frontend — handling the AI career recommendation call:

```
const handleSubmit = async () => { setLoading(true);
```

```
  try {
    const res = await axios.post(BACKEND + "/career-recommend", form);
    const jsonMatch = res.data.data.match(/\[[\s\S]*\]/);
    if (jsonMatch) { setCareers(JSON.parse(jsonMatch[0])); setPage("result"); }
    else alert("Could not parse results. Try again!");
  } catch { alert("Error connecting to backend!"); }
  setLoading(false);
};
```